

Recuperatorio del Parcial 1 — 2025

Sistemas Operativos

Tomás Rodeghiero

UNRC — Universidad Nacional de Río Cuarto

2025

Resumen

Resolución completa del Recuperatorio del Parcial 1 del año 2025 de la cátedra de Sistemas Operativos (UNRC). El parcial consta de cinco ejercicios: (i) descripción detallada del flujo de una interrupción/excepción y el rol del *trapframe*; (ii) implementación en pseudo-código de un lock reentrante (que evita *self-lock* cuando el mismo dueño lo invoca repetidamente); (iii) comparación entre los algoritmos de planificación FIFO y Round Robin; (iv) verdadero/falso sobre paginación, segmentación, *copy-on-write* y *mmap*; (v) cálculos sobre una arquitectura de 32 bits con páginas de 4 KB, incluyendo construcción de la tabla de páginas de un proceso y determinación de page faults. Cada ejercicio se desarrolla con la estructura **Enunciado** → **Análisis** → **Desarrollo** → **Respuesta**.

Índice

1. Ejercicio 1 — Interrupciones, excepciones y <i>trapframe</i> (2 ptos)	1
2. Ejercicio 2 — Lock reentrante sin <i>selflock</i> (2 ptos)	4
3. Ejercicio 3 — FIFO vs Round Robin (2 ptos)	6
4. Ejercicio 4 — V/F sobre memoria virtual (2 ptos)	7
5. Ejercicio 5 — Paginación: tabla y page faults (2 ptos)	9
Resumen del recuperatorio	11

1. Ejercicio 1 — Interrupciones, excepciones y *trapframe* (2 ptos)

Enunciado. Describa de forma general los pasos que sigue la CPU y el código de un SO en una interrupción o excepción hasta su retorno. En particular, describa el rol del *trapframe*.

Análisis: qué es un *trap*

Una **trap** es cualquier evento que interrumpe la ejecución normal de un proceso y transfiere el control al kernel. Hay tres familias:

- **Interrupción:** viene de un dispositivo externo (timer, disco, red, teclado). Es *asíncronica*: ocurre en cualquier punto de la ejecución.
- **Excepción:** se origina en la propia instrucción que está ejecutando la CPU (page fault, división por cero, instrucción ilegal, acceso a memoria fuera de rango). *Síncronica* con la ejecución.
- **Trap explícita / syscall:** una instrucción especial (`syscall` en x86, `ecall` en RISC-V, `svc` en ARM) que el proceso ejecuta para pedir al kernel un servicio.

Las tres usan el *mismo* mecanismo de hardware: la CPU detecta el evento, salva el estado mínimo y salta a un punto fijo del kernel.

Flujo completo paso a paso

Fase 1 — Hardware atiende el trap.

1. Si las interrupciones están habilitadas, la CPU acepta el evento.
2. **Salva en registros especiales** el estado mínimo: el PC actual (para saber a dónde volver), el modo de ejecución previo (user/kernel), y la causa del trap (**scause** en RISC-V, vector number en x86).
3. Cambia a **modo supervisor** si no estaba ya.
4. Salta a una dirección fija del kernel definida en el *trap vector* (**stvec** en RISC-V, IDT en x86, VBAR en ARM).

Fase 2 — Trampolín y guardado completo del contexto. El primer código que se ejecuta del kernel es un pequeño *trampolín* en assembly (**uservec** en xv6). Su tarea es:

1. Conmutar a un stack del kernel (no del proceso).
2. Salvar **todos los registros** del CPU (de propósito general, de estado, etc.) en una estructura llamada **trapframe** en el stack del kernel.
3. Cargar la tabla de páginas del kernel (si era una trap desde modo usuario).
4. Saltar a una función C principal (e.g., **usertrap**).

Fase 3 — Despacho según la causa. La función C principal lee el campo de causa (e.g., **scause**) y despacha al *handler* correspondiente:

- Syscall → **syscall()**, que extrae el número de syscall del trapframe y llama a la función correspondiente (**sys_read**, **sys_write**, **sys_fork**, etc.).
- Page fault → **page_fault_handler()**.
- Timer / IRQ de disco → ISR del driver correspondiente.
- Excepción no recuperable (división por cero, etc.) → enviar **SIGSEGV/SIGFPE** al proceso.

Fase 4 — Posible cambio de proceso. Después del manejo, el kernel puede invocar al scheduler:

- Si fue interrupción del timer y el proceso usó su quantum → **yield()**: el scheduler elige otro proceso. Cambio de contexto.
- Si fue un syscall bloqueante (**read** sobre disco): el proceso pasa a **SLEEPING** y el scheduler elige otro.

Eventualmente, el scheduler vuelve a planificar el proceso original (en algún momento futuro).

Fase 5 — Retorno. Cuando es momento de regresar al proceso interrumpido:

1. Se carga la tabla de páginas del proceso.
2. Se ejecuta el trampolín de retorno (**userret** en xv6), que restaura todos los registros desde el trapframe.
3. Se ejecuta la instrucción atómica de retorno (**sret** en RISC-V, **iret** en x86, **eret** en ARM). Esta instrucción restaura el PC desde **sepc**, el modo a usuario y rehabilita las interrupciones, todo en un solo paso.

El proceso reanuda como si nada hubiera pasado.

El rol del *trapframe*

El *trapframe* es una **estructura de datos en el stack del kernel** del proceso, asociada al PCB, que conserva el estado completo del proceso al momento de la trap.

Estructura típica (xv6/RISC-V):

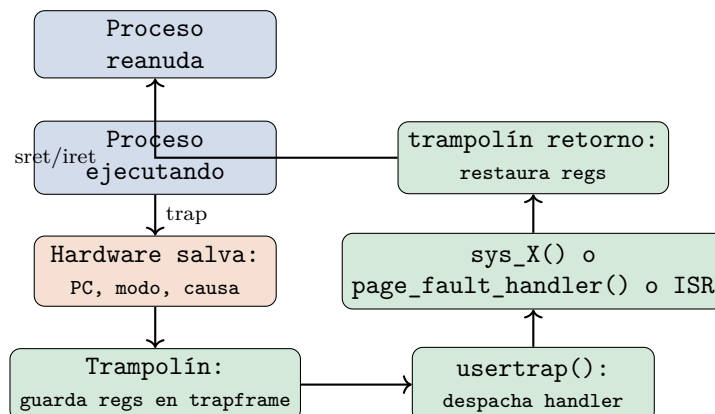
```
struct trapframe {
    uint64 kernel_satp; // dir. tabla de paginas del kernel
    uint64 kernel_sp; // stack pointer del kernel
    uint64 kernel_trap; // direccion de usertrap()
    uint64 epc; // PC en el momento del trap (sepc)
    uint64 kernel_hartid; // CPU actual

    /* Registros de proposito general */
    uint64 ra, sp, gp, tp;
    uint64 t0, t1, t2, t3, t4, t5, t6;
    uint64 s0, s1, ..., s11;
    uint64 a0, a1, ..., a7;
};
```

Funciones del *trapframe*:

1. **Salvar el estado del proceso:** cuando ocurre un trap, el trampolín copia todos los registros al *trapframe*. Esto permite que el handler del kernel use los registros libremente sin perder el estado del proceso.
2. **Pasar argumentos al handler:** en una syscall, los argumentos del usuario están en registros (a0-a7 en RISC-V). El handler los lee del *trapframe*.
3. **Devolver resultados al proceso:** para que una syscall retorne un valor, el handler escribe el resultado en el campo correspondiente del *trapframe* (e.g., a0). Cuando el trampolín de retorno restaura los registros, el proceso “ve” el resultado en su registro a0.
4. **Soportar cambios de contexto:** si el handler decide cambiar de proceso, el *trapframe* del proceso saliente queda guardado; al volver a planificarlo, se restaura desde su propio *trapframe*.
5. **Auditoría / debugging:** el contenido del *trapframe* contiene exactamente la *imagen registral* del proceso en el momento de la trap, útil para diagnóstico (e.g., para reportar el PC donde ocurrió un page fault).

Diagrama del flujo



Respuesta

Pasos generales: (1) la CPU detecta el trap, salva un estado mínimo (PC, modo, causa) y salta al *trap vector*; (2) un trampolín en assembly del kernel guarda todos los registros del proceso en el **trapframe** (estructura asociada al PCB) y cambia al stack del kernel; (3) una función C en el kernel despacha al *handler* adecuado según la causa; (4) el handler procesa (eventualmente cambia de contexto); (5) al volver, el trampolín de retorno restaura los registros desde el trapframe y ejecuta la instrucción atómica de retorno (**sret/iret/eret**) que devuelve el control al proceso en modo usuario.

Rol del trapframe: es el “snapshot” completo del estado del proceso al ocurrir el trap. Sirve para (a) que el handler del kernel pueda usar los registros sin perder el contexto del proceso, (b) leer los argumentos de una syscall, (c) escribir el resultado de la syscall (que será visible al proceso cuando reanude), y (d) soportar cambios de contexto: cada proceso tiene su propio trapframe asociado a su PCB.

2. Ejercicio 2 — Lock reentrante sin *selflock* (2 ptos)

Enunciado. Un proceso o el kernel produce un *selflock* si invoca a `lock(&lk)` dos veces sin que en el medio haya un `unlock(&lk)`. Proponga una implementación en pseudo-código de `lock/unlock` que permita invocar repetidamente a `lock(&lk)` sin que se produzca un *selflock*. Asuma que existen `cpuid()` y `pid()` que retornan el ID de la CPU y el PID actual.

Análisis: lock reentrante (recursive lock)

La idea es identificar al *dueño actual* del lock por la combinación (`cpuid`, `pid`) y mantener un **contador**:

- Si el lock está libre, lo tomamos normalmente y guardamos nuestra identidad.
- Si el lock está tomado y *nosotros somos el dueño*, simplemente incrementamos el contador y retornamos sin bloquearnos.
- Si el lock está tomado por *otro*, esperamos (*spin*) como un lock normal.
- `unlock` decrementa el contador. Solo cuando llega a 0 libera realmente el lock.

Esto convierte el lock en **reentrante**: el mismo dueño puede tomarlo varias veces de forma anidada sin *selflock*.

Pseudo-código

```
typedef struct {
    int locked; /* 0 = libre, 1 = tomado */
    int owner_cpu; /* CPU del dueño actual */
    int owner_pid; /* PID del dueño actual */
    int depth; /* veces que el mismo dueño lo tomo */
} lock_t;

void lock(lock_t *lk) {
    int my_cpu = cpuid();
    int my_pid = pid();

    /* 1) Si YA somos los dueños, simplemente incrementamos. */
    if (lk->locked == 1 &&
        lk->owner_cpu == my_cpu &&
        lk->owner_pid == my_pid) {
        lk->depth++;
        return;
    }
}
```

```

    /* 2) No lo tenemos: intentamos adquirirlo atómicamente. */
    while (atomic_swap(&lk->locked, 1) == 1) {
        /* Si lo tenía otro dueño, seguimos esperando.
         * Si lo tenía el mismo dueño, no llegamos a este while
         * (volvimos en el if anterior). */
        ; /* spin-wait */
    }

    /* 3) Lo conseguimos: marcamos nuestra identidad. */
    lk->owner_cpu = my_cpu;
    lk->owner_pid = my_pid;
    lk->depth = 1;
}

void unlock(lock_t *lk) {
    int my_cpu = cpuid();
    int my_pid = pid();

    /* Verificación de coherencia. */
    if (lk->locked != 1 ||
        lk->owner_cpu != my_cpu ||
        lk->owner_pid != my_pid) {
        panic("unlock por un thread que no es el dueño");
    }

    lk->depth--;
    if (lk->depth == 0) {
        lk->owner_cpu = -1;
        lk->owner_pid = -1;
        lk->locked = 0; /* liberación atómica */
    }
    /* Si depth > 0, todavía hay locks anidados del mismo dueño;
     * el lock NO se libera. */
}

```

Por qué funciona

- **Sin selflock:** si el mismo (cpu, pid) llama dos veces a lock, la segunda invocación cae en el primer if y simplemente incrementa depth. No espera en el while.
- **Exclusión mutua con otros:** si otro (cpu, pid) intenta tomar, ve locked = 1 con dueño distinto, no entra al if y se va al while, donde espera.
- **Liberación correcta:** cada lock hace depth++, cada unlock hace depth-. Solo cuando depth llega a 0 se libera locked de verdad.

Ejemplo de uso

```

void funcion_externa() {
    lock(&lk); /* depth pasa a 1 */
    funcion_interna();
    unlock(&lk); /* depth pasa a 0, libera */
}

void funcion_interna() {
    lock(&lk); /* SAME owner; depth pasa a 2; no se bloquea */
    /* ... */
    unlock(&lk); /* depth pasa a 1 */
}

```

Sin lock reentrante, `lock(&l)` dentro de `funcion_interna` se bloquearía contra sí misma (*self-lock*). Con el lock reentrante propuesto, funciona correctamente.

Respuesta

La clave es identificar al *dueño actual* con (`cpuid`, `pid`) y mantener un contador `depth`. `lock` verifica primero si el llamante ya es dueño; si lo es, incrementa `depth` y retorna. `unlock` decrementa y solo libera realmente cuando `depth` llega a 0.

Importante: la operación de adquisición debe ser atómica (`atomic_swap`, `cmpxchg`, LL/SC) para garantizar exclusión mutua con otros dueños en un multiprocesador.

3. Ejercicio 3 — FIFO vs Round Robin (2 ptos)

Enunciado. Describa la diferencia entre el algoritmo de planificación FIFO y Round Robin.

FIFO (First In First Out, también FCFS)

Idea. Una sola cola **READY**. Los procesos se encolan al llegar; el primero en arribar es el primero en ejecutar. Una vez asignada la CPU, el proceso **ejecuta hasta terminar o bloquearse** (no preemption).

Propiedades:

- **No-preemptivo:** ningún proceso es desalojado por agotar tiempo.
- Implementación: una cola simple FIFO.
- Métricas: turnaround promedio depende fuertemente del orden de llegada y de los tamaños de las ráfagas.

Convoy effect. Si un proceso CPU-bound largo llega primero, todos los que vienen detrás esperan hasta que termine, aunque sean cortos. El tiempo de espera promedio crece mucho.

Diagrama de Gantt típico (P1: 10, P2: 1, P3: 1, en orden de llegada):



P2 y P3 esperan ~ 10 unidades aunque solo necesitaran 1. Turnaround promedio: $(10 + 11 + 12)/3 \approx 11$.

Round Robin

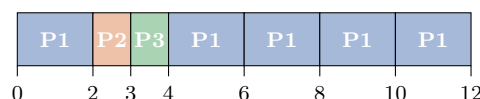
Idea. Una cola **READY**, pero a cada proceso se le asigna un **quantum** (*time slice*, q). Si al expirar el quantum el proceso aún no terminó, se lo **desaloja** (preempt) y se reencola al final. El scheduler toma el siguiente.

Propiedades:

- **Preemptivo:** el timer fuerza el desalojo al expirar el quantum.
- Implementación: cola FIFO + temporizador.
- Métricas: garantiza un tiempo de respuesta acotado por $N \times q$ (con N procesos en **READY**).

Sin convoy effect. Aunque haya un proceso largo, los demás obtienen CPU cada $\leq N \cdot q$ unidades. Bueno para sistemas interactivos.

Diagrama de Gantt típico (mismos procesos, $q = 2$):



P2 termina en $t = 3$ y P3 en $t = 4$, muchísimo mejor que con FIFO. P1 termina al final, pero no hace esperar a los demás.

Comparación

Característica	FIFO (FCFS)	Round Robin
Preemptivo	No	Sí (al expirar quantum)
Estructura	cola FIFO	cola FIFO + timer
Tiempo de respuesta	Malo (varía mucho)	Acotado por $N \cdot q$
Convoy effect	Sufre	Lo evita
Cambios de contexto	Pocos	Muchos (cada quantum)
Apto para interactivos	No	Sí
Apto para batch	Sí	Solo si q es grande
Caso $q \rightarrow \infty$	—	Degenera en FIFO
Caso $q \rightarrow 0$	—	Overhead domina

Respuesta

Diferencia esencial: FIFO es **no preemptivo** y ejecuta los procesos en estricto orden de llegada hasta que terminen. Round Robin es **preemptivo** y le asigna a cada proceso un cuanto de tiempo (*quantum*); al expirar el quantum, el proceso vuelve al final de la cola.

Consecuencia práctica: RR evita el *convoy effect* y garantiza tiempo de respuesta acotado, ideal para sistemas interactivos. FIFO es más simple y tiene menos cambios de contexto, ideal para sistemas *batch* con cargas predecibles. El *quantum* de RR es el parámetro que regula el compromiso respuesta vs. overhead: si $q \rightarrow \infty$, RR se vuelve FIFO; si $q \rightarrow 0$, el overhead de cambios de contexto domina.

4. Ejercicio 4 — V/F sobre memoria virtual (2 ptos)

4.a — “El paginado define espacios lógicos a cada proceso; la segmentación no”

Respuesta

F. Ambas técnicas, paginado y segmentación, permiten definir un espacio lógico de direcciones para cada proceso. La diferencia es la *unidad* de mapeo:

- **Paginado:** el espacio se divide en páginas de tamaño fijo. La tabla de páginas mapea vpn a ppn.
- **Segmentación:** el espacio se divide en segmentos de tamaño variable (código, datos, stack, etc.) con base y límite. La tabla de descriptores (GDT/LDT en x86) mapea cada segmento a su ubicación física.

Lo correcto sería que *ambas* ofrecen aislamiento entre procesos. De hecho, x86 históricamente usó segmentación *antes* de tener paginado. La segmentación pura cayó en desuso (es menos eficiente para memoria virtual; ver ítem b), pero *sí* provee espacios lógicos.

4.b — “El paginado es más adecuado que la segmentación para memoria virtual”

Respuesta

V. La paginación tiene varias ventajas críticas para memoria virtual:

- **Tamaño uniforme:** todas las páginas miden lo mismo. El swap-in/swap-out es de tamaño fijo, simple y predecible.
- **Sin fragmentación externa:** cualquier frame físico libre sirve. En segmentación, segmentos de tamaño variable producen huecos no aprovechables.
- **Granularidad fina:** con *demand paging*, se carga solo lo que se accede. Con segmentación, habría que traer un segmento entero (puede ser MB).
- **TLB y MMU simples:** el hardware de paginado es más sencillo y rápido que el de segmentación + paginado combinado.

Por eso todos los SO modernos usan **paginado** (o paginado combinado con segmentación residual, como x86), y los segmentos “puros” son cosa del pasado.

4.c — “COW puede usarse para implementar fork() eficientemente”

Respuesta

V. Es exactamente la motivación de *copy-on-write*. La implementación de `fork()` con COW es:

1. Al hacer `fork()`, el kernel *no copia* las páginas del padre. En cambio, crea para el hijo una nueva tabla de páginas que apunta a los mismos *frames físicos* del padre.
2. Las páginas se marcan como **solo lectura** en ambas tablas (padre e hijo).
3. Mientras ambos solo lean, comparten físicamente (gratis).
4. Cuando uno *escribe* a una página compartida, el hardware levanta un *page fault*. El kernel atiende: copia la página afectada a un nuevo frame, actualiza la tabla de quien escribió para que apunte ahí (writable), y deja al otro con la copia original.

Por qué es eficiente: en el caso típico (`fork + exec`), el hijo no escribe nada del padre antes de `exec()`, así que NUNCA se copia ninguna página. El costo de `fork` se reduce a copiar la tabla de páginas y unos metadatos, no varios MB de RAM.

4.d — “mmap() puede usarse para implementar exec() eficientemente”

Respuesta

V. Esto es exactamente lo que hacen los SO modernos. La implementación de `exec()` con `mmap`:

1. En vez de leer todo el ejecutable a memoria, `exec` hace `mmap` del archivo ejecutable en el espacio de direcciones del proceso.
2. Las páginas se marcan como “presentes en disco” pero no en RAM ($V = 0$ en la `pte`, con información en otro campo de dónde leer).
3. Cuando el proceso empieza a ejecutar instrucciones, cualquier acceso a una página todavía no cargada produce un *page fault*.
4. El kernel atiende el fault leyendo *esa* página del disco.

Por qué es eficiente:

- Solo se lee del disco lo que efectivamente se ejecuta o accede. Muchas funciones del binario nunca se llaman, y sus páginas nunca se cargan.
- Si varias instancias del mismo binario corren en paralelo, las páginas de `.text` (read-only) se comparten físicamente.
- El kernel administra una *page cache* unificada; las páginas leídas pueden quedar en RAM aunque otro proceso cierre el binario, facilitando que el siguiente `exec()` del mismo programa sea instantáneo.

5. Ejercicio 5 — Paginación: tabla y page faults (2 ptos)

Enunciado. En una arquitectura de 32 bits, espacio de direcciones de 4 GB y páginas de 4 KB, un proceso mapeado desde la dirección 0 tiene:

- 7 KB de código.
- 3.5 KB de datos globales.
- 2 páginas de stack a partir de `0x7fe000`.

(a) Dar el esquema de la tabla de páginas del proceso. (b) ¿La dirección `0x381A` produce page fault? (c) Ídem para `0x7ff100`.

Análisis de parámetros

- Página: $4\text{ KB} = 2^{12} = 0x1000$ bytes.
- Espacio total: $4\text{ GB} = 2^{32}$ bytes.
- Total de páginas posibles: $2^{32} \div 2^{12} = 2^{20} = 1\,048\,576$ páginas. La tabla tiene 2^{20} entradas (típicamente representadas como un árbol multinivel para no ocupar 4 MB por proceso).
- Direcciones virtuales: 20 bits altos = vpn, 12 bits bajos = offset.

Layout del proceso

Páginas por área (con padding a frontera de página, como hace el loader ELF):

- **Código** (7 KB): $\lceil 7/4 \rceil = 2$ páginas $\rightarrow$ vpn 0 y 1 (rango virtual `[0x0000, 0x2000)`).
- **Datos globales** (3.5 KB): $\lceil 3.5/4 \rceil = 1$ página $\rightarrow$ vpn 2 (rango `[0x2000, 0x3000)`).
- **Stack**: 2 páginas a partir de `0x7fe000`. Como cada página es `0x1000`, las páginas son $\text{vpn} = 0x7fe000 / 0x1000 = 0x7fe = 2046$ y $\text{vpn} = 0x7ff = 2047$.

5.a — Esquema de la tabla de páginas

vpn (hex)	rango virtual	V	R/W/X	Área	Notas
0x000	[0x0000, 0x1000)	1	R + X	código pág. 1	4 KB
0x001	[0x1000, 0x2000)	1	R + X	código pág. 2	3 KB + 1 KB pad
0x002	[0x2000, 0x3000)	1	R + W	datos globales	3.5 KB + 0.5 KB pad
0x003–0x7fd	[0x3000, 0x7fe000)	0	—	gap inválido	2043 páginas
0x7fe	[0x7fe000, 0x7ff000)	1	R + W	stack pág. 1	
0x7ff	[0x7ff000, 0x800000)	1	R + W	stack pág. 2	
$\geq 0x800$	[0x800000, ...)	0	—	inválidas	resto

Solo **5 entradas válidas** (vpn 0, 1, 2, 0x7fe, 0x7ff) de 2^{20} posibles. El resto está inválido ($V = 0$). Esto justifica por qué la tabla se implementa como un árbol multinivel: solo los nodos correspondientes a regiones efectivamente usadas se reservan en RAM.

Diagrama del espacio de direcciones

vpn 0: código	0x00000000
vpn 1: código (+pad)	0x00001000
vpn 2: datos (+pad)	0x00002000 0x00003000
GAP vpn 3 a 2045 ([0x3000, 0x7fe000))	
vpn 2046: stack	0x007fe000
vpn 2047: stack	0x007ff000 0x00800000

5.b — ¿0x381A produce page fault?

Descomposición (20 bits vpn, 12 bits offset):

$0x381A$ en 32 bits = $0x0000381A$

$0x381A \div 0x1000 = 3$ (cociente)

$0x381A \bmod 0x1000 = 0x81A$

Por tanto: $\text{vpn} = 3$, $\text{offset} = 0x81A$.

Consulta a la tabla: la entrada $\text{vpn} = 3$ cae en el gap inválido (entre vpn 3 y vpn 2045). $V = 0$.

Respuesta

(5.b) SÍ produce page fault. La dirección $0x381A$ está en $\text{vpn} = 3$, primera página después del área de datos. Esa página no está mapeada ($V = 0$). El hardware genera un page fault y el kernel envía SIGSEGV al proceso.

5.c — ¿0x7ff100 produce page fault?

Descomposición:

$0x7ff100 \div 0x1000 = 0x7ff = 2047$

$0x7ff100 \bmod 0x1000 = 0x100$

Por tanto: $\text{vpn} = 0x7ff = 2047$, $\text{offset} = 0x100$.

Consulta a la tabla: la entrada `vpn = 0x7ff` corresponde a la **segunda página de stack**. Está válida ($V = 1, R + W$).

Respuesta

(5.c) NO produce page fault. La dirección `0x7ff100` está en `vpn = 0x7ff = 2047`, que es la *segunda página del stack* del proceso (la que arranca en `0x7ff000`). Está mapeada, con permisos $R+W$. El acceso es legítimo.

Verificación numérica

- Stack comienza en `0x7fe000` (`vpn 0x7fe`). $0x7fe \cdot 0x1000 = 0x7fe000$. ✓
- Stack ocupa 2 páginas: `vpn 0x7fe` y `vpn 0x7ff`, rango total `[0x7fe000, 0x800000)`.
- `0x7ff100` cae claramente dentro de ese rango. ✓

Resumen del recuperatorio

#	Tema	Respuesta sintética
1	Traps + trapframe	Hardware salva estado mínimo, trampolín guarda regs en trapframe, dispatch a handler C, retorno con <code>sret</code> . Trapframe = snapshot completo del estado del proceso
2	Lock reentrante	Guardar (<code>cpu, pid</code>) del dueño + contador <code>depth</code> ; si soy el dueño, incremento sin bloquear; <code>unlock</code> decrementa y libera al llegar a 0
3	FIFO vs RR	FIFO: no preemptivo, sufre convoy; RR: preemptivo con quantum, evita convoy, mejor para interactivos a costa de overhead de cambios de contexto
4	V/F memoria	a:F (segmentación también define espacios), b:V (uniformidad, no frag. externa), c:V (COW), d:V (mmap del ejecutable con demand paging)
5	Paginación num.	(a) 5 entradas válidas: <code>vpn 0,1,2 + 2046, 2047</code> ; (b) <code>0x381A vpn=3</code> (gap) → page fault; (c) <code>0x7ff100 vpn=2047</code> (stack) → NO fault